

Report 2 - SF2957 Statistical Machine Learning

Vincent Hallberg [REDACTED] Ville Wassberg [REDACTED]
Klara Zimmerman [REDACTED] Noel Yonatan Efrem [REDACTED]

December, 2024

Introduction

In this project we aim to use reinforcement learning to train an agent to play blackjack as well as possible. The agent will be trained by trying to maximise the reward with respect to its actions, stay or hit. The rewards are defined as follows

- **blackjack:** Reward, $R_t = 1.5$,
- **higher score than dealer and not bust:** Reward, $R_t = 1.0$,
- **Dealer bust and agent is not:** Reward, $R_t = 1.0$.

The state spaces considered in this project are the following:

- **Extended state-space :** Representing the agent's set of cards (the cards are known) and the dealer's sum:

$$\mathcal{S}_1 = \{(s_1, s_2, \dots, s_{10}, S_d) : \sum_{i=1}^{10} s_i \leq 31, S_i \geq 0 \text{ for } i = 1, \dots, 10, \text{ and } S_d \leq 26\},$$

where S_d is the dealer's card sum and $(s_1^a, \dots, s_{10}^a)$ is the dealer's hand.

- **Non-informative state-space:** Representing only the agent's and the dealer's sums without knowing the specific cards:

$$\mathcal{S}_2 = \{(S_a, u, S_d) : 1 \leq S \leq 31, u \in \{0, 1\}, 1 \leq S_d \leq 26\}.$$

The lesser informative state space is flawed since it can not be seen whether a hand is a black jack or not, and it does not keep count of the remaining number of cards either. However, both state spaces will be considered in the implementation.

Both *On-policy Monte Carlo learning* and *Q-learning* is implemented and evaluated when training the agent. Although, the MC will only be implemented to the extended space $\mathcal{S}_1$ while both $\mathcal{S}_1, \mathcal{S}_2$ are considered for Q-learning. Furthermore, the learning rate for the Q-learning algorithm will be tuned by exploring the decay rate ω , where the learning rate is defined as $\alpha_n = \text{constant} \cdot n^{-\omega}$, $0 \leq \omega < 1$. Also, the impact of decaying exploration for the ϵ -greedy algorithm is compared to the constant rate. A comparison of the models' learning performances will be done by evaluating their average reward.

Finally, action tables of the close to optimal strategy for the agent is displayed, both for when there is a usable ace in the agent's hand and when it is not. The action tables describes whether the agent should hit or stay depending on the hands of the agent and the dealer.

Mathematical background

On-policy Monte Carlo learning

On-policy learning uses the same policy for both decision-making and policy updating. In on-policy Monte Carlo (MC) learning, we simulate from the transition probabilities $p(s', r|s, a)$ without knowing them. We simulate entire episodes (complete games), and compute the cumulative reward from each visited state-action pair in the episode. This cumulative reward, so-called return, is then used to update the action-value function $q_i(s, a)$. Then the policy π is updated in an ϵ -greedy manner based on the current $q_i(s, a)$. This process iterates until the policy converges.

Updating the action-value function

After each episode i , the action-value function is updated using

$$q_i(s, a) = q_{i-1}(s, a) + \frac{1}{N_i(s, a)} \left(G_{s,a}^{(i)} - q_{i-1}(s, a) \right),$$

where $N_i(s, a)$ is the number of times state-action pair (s, a) has been visited up until episode i .

The return $G_{s,a}^{(i)}$ is the total discounted reward obtained from state s , taking action a and following the policy π thereafter. It is computed as:

$$G_{s,a}^{(i)} = \sum_{k=0}^{\tau^{(i)} - \tau_{s,a}^{(i)} - 1} \gamma^k R_{\tau_{s,a}^{(i)} + k + 1}^{(i)},$$

where γ is the discount factor, R is the reward, $\tau_{s,a}^i$ is the time step at which (s, a) was visited during episode i and τ^i is the terminal time.

Update of Average Reward

The average reward is updated by computing the running arithmetic mean,

$$m_n = m_{n-1} + \frac{1}{n} (x_n - m_{n-1}),$$

where n is the number of episodes and x_n the reward. By the law of large numbers, this converges to the expected reward.

ϵ -greedy Policy update

The policy is updated using the ϵ -greedy algorithm, which assigns probabilities as follows:

$$\pi_i(a \mid s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(s)|}, & \text{if } a = a^*(s), \\ \frac{\epsilon}{|A(s)|}, & \text{if } a \neq a^*(s), \end{cases} \quad (1)$$

where $A(s)$ is the set of all possible actions in state s , and the optimal action is selected by $a^*(s) = \arg \max_{a \in A(s)} q_{i-1}(s, a)$. That is, with a probability of $1 - \epsilon$, the agent follows a greedy action, and otherwise takes a random action.

The following pseudo-code shows our implementation of the described updates.

Pseudo-code for Updating Average Reward and Action-Values in On-policy MC algorithm

1. Update avg_reward:

$$\text{avg_reward} \leftarrow \text{avg_reward} + \frac{1}{\text{episode}} \cdot (\text{episode_reward} - \text{avg_reward})$$

2. Compute $G(s, a)$ for first visits:

For each index i , (state, action, reward) in episode_track:

If (state, action) is visited for the first time:

Set $\tau^{(i)} = \text{length}(\text{episode_track})$

Set $\tau_{s,a}^{(i)} = i$

Compute $G[\text{state}][\text{action}]$ as:

$$G[\text{state}][\text{action}] \leftarrow \sum_{k=0}^{\tau^{(i)} - \tau_{s,a}^{(i)} - 1} \gamma^k \cdot \text{reward}[k + 1 + \tau_{s,a}^{(i)}]$$

3. Update the Q-function for each visited state/action pair:

For each state in episode_state_action_count:

For each action in range(env.action_space.n):

If episode_state_action_count[state][action] > 0:

$$Q[\text{state}][\text{action}] \leftarrow Q[\text{state}][\text{action}] + \frac{G[\text{state}][\text{action}] - Q[\text{state}][\text{action}]}{\text{state_action_count}[\text{state}][\text{action}]}$$

Q-learning

Q-learning is an off-policy reinforcement learning method, where the agent learns the value of an optimal policy independently of the behavioral policy. The target values are computed using an optimal policy where the action is chosen through maximizing the reward in each iteration. Meanwhile, the agent follows an ϵ -greedy behavior policy for exploration, as presented in (1).

We have that the action-value function, also called Q-value, for a policy π is:

$$q_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right].$$

which intuitively can be understood as the expected cumulative reward obtained given one takes an action a in state s and then following policy π . For the optimal policy π_* , the optimal action value function $q_*(s, a)$ satisfies the Bellman optimality equation:

$$q_*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a \right].$$

Q-learning is an iterative method that approximates $q_*(s, a)$ using the Bellman optimality equation and then update rule is:

$$q(S_{t-1}, A_{t-1}) = q(S_{t-1}, A_{t-1}) + \alpha \left[R_t + \gamma \sup_{a \in \mathcal{A}} q_{t-1}(S_t, a) - q(S_{t-1}, A_{t-1}) \right],$$

where:

- $q(S_{t-1}, A_{t-1})$: The current estimate of the Q-value for state S_{t-1} and action A_{t-1} .
- α : The learning rate, which controls the step size of updates.
- R_t : The reward observed after taking action A_{t-1} in state S_{t-1} .
- $\sup_{a \in \mathcal{A}} q_{t-1}(S_t, a)$: The maximum Q-value for the next state S_t over all possible actions $a \in \mathcal{A}$, using the Q-values from the previous time step.

The steps taken in Q-learning are:

1. Given the action value q_{t-1} , select the action A_{t-1} using, for instance, the ϵ -greedy policy as defined in (1).
2. Execute A_{t-1} and observe the reward R_t and the next state S_t .
3. Update the Q-value using the update rule above.

Compared to the Bellman based iterative policy evaluation given in (27) in course literature, the Q-learning method does not require the transition probability $p(s', r|s, a)$.

The following pseudo-code demonstrates our implementation of the updates for Q-learning.

Pseudo-code for Computing the Learning Rate and Updating the Action-values in Q-learning algorithm

1. Update the learning rate according to the number of times that the particular current state-action pair has been seen before:

$$\omega = \text{decay rate}$$

$$c = 1$$

$$n = \text{state_action_count}[state][action]$$

$$\alpha = c \times n^{-\omega}$$

2. Update the Q-function, where the target represents the future award assuming an optimal policy, regardless of the action actually taken:

$$Q[state][action] \leftarrow Q[state][action] + \alpha * (\text{action_reward} + \gamma * \max(Q[state2]) - Q[state][action])$$

Results

Initial Results

After implementing the On-policy MC and Q-learning algorithms, we plot the average reward over number of episodes. We let the number of simulated episodes equal 50000 in order to observe convergence. Initially, we are given the following parameters

- Exploration rate $\epsilon = 0.05$ (constant),
- Learning rate $\alpha_n = cn^{-\omega}$ with $c = 1$, $\omega = 0.77$,
- Discount factor $\gamma = 1$.

The result of the average rewards obtained from these simulations are plotted in figures 1 and 2, where we vary the number of card decks used in the game.

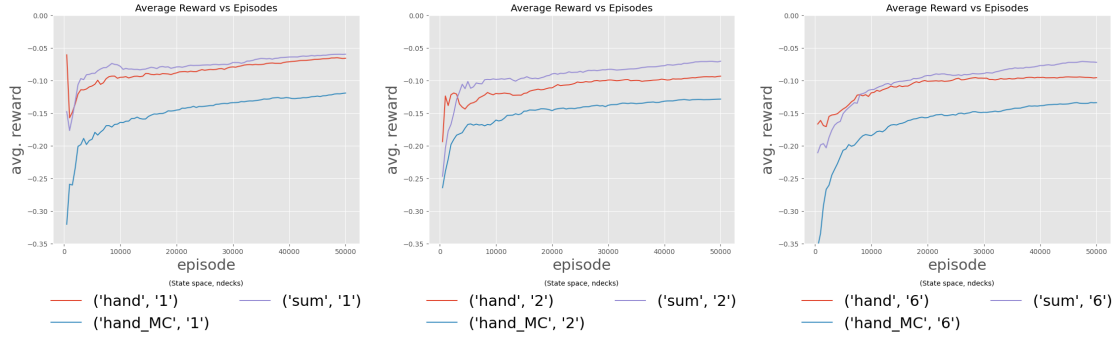


Figure 1: Results of MC (state-space S_1) and Q-learning (state-space S_1 and S_2) trained with 1, 2 and 6 decks using given parameter values: $\epsilon = 0.05, \omega = 0.77, \gamma = 1$.

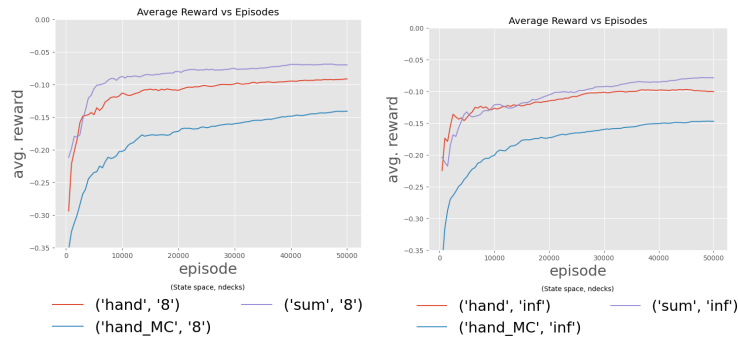


Figure 2: Results of MC (state-space S_1) and Q-learning (state-space S_1 and S_2) trained with 8 and infinite many decks using given parameter values: $\epsilon = 0.05, \omega = 0.77, \gamma = 1$.

From these plots, we can conclude that the Q-learning algorithm performs better across all deck sizes using the initial parameter values. Furthermore, using the lesser informative state space S_2 outperforms the extended state space S_1 using these initial parameters.

The number of decks used yields different average rewards. The agent seems to learn the best strategy when playing with only one deck of cards. This goes for all three methods.

Q-learning with varying learning rates

Next, we study the effect of the learning rate $\alpha_n = n^{-w}$ in the Q-learning algorithm by varying the decay rate ω . We plot the results for each deck-size, using the values $\omega \in \{0, 0.25, 0.75, 0.99\}$. The results are shown in figures 3, 4, 5, 6 and 7.

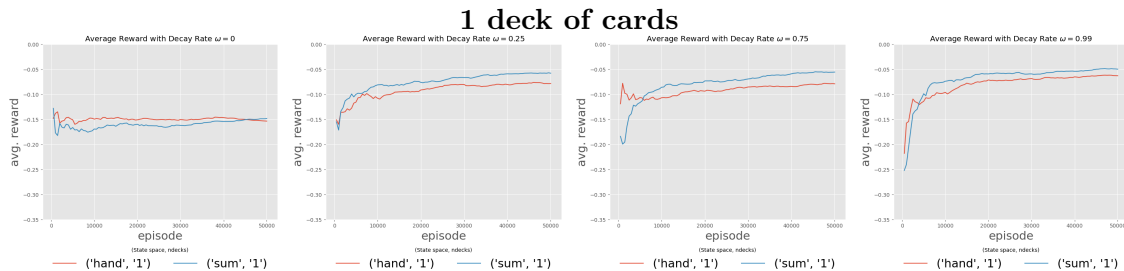


Figure 3: Q-learning with one deck, and decay rates $\omega \in \{0, 0.25, 0.75, 0.99\}$. Best model: State-space S_2 ("sum") with decay rate $\omega = 0.99$.

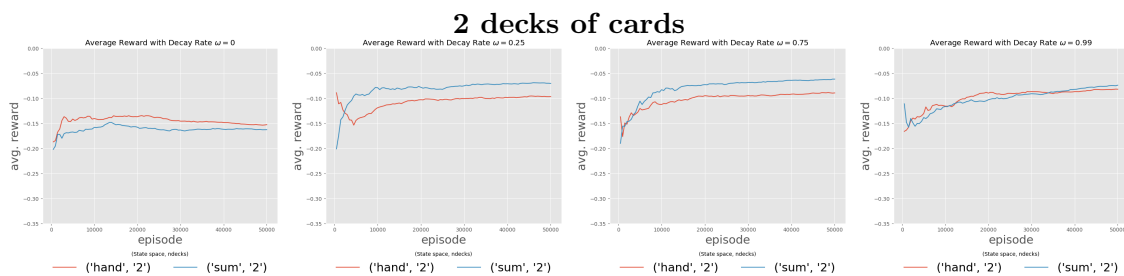


Figure 4: Q-learning with two decks, and decay rates $\omega \in \{0, 0.25, 0.75, 0.99\}$. Best model: State-space S_2 ("sum") with decay rate $\omega = 0.75$.

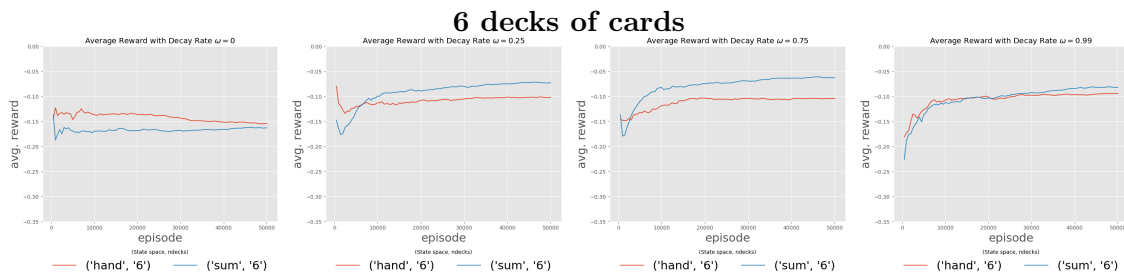


Figure 5: Q-learning with six decks, and decay rates $\omega \in \{0, 0.25, 0.75, 0.99\}$. Best model: State-space S_2 ("sum") with decay rate $\omega = 0.75$.

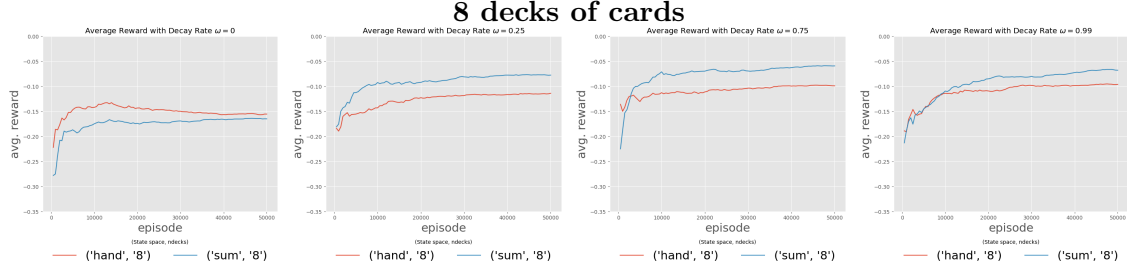


Figure 6: Q-learning with eight decks, and decay rates $\omega \in \{0, 0.25, 0.75, 0.99\}$. Best model: State-space S_2 ("sum") with decay rate $\omega = 0.75$.

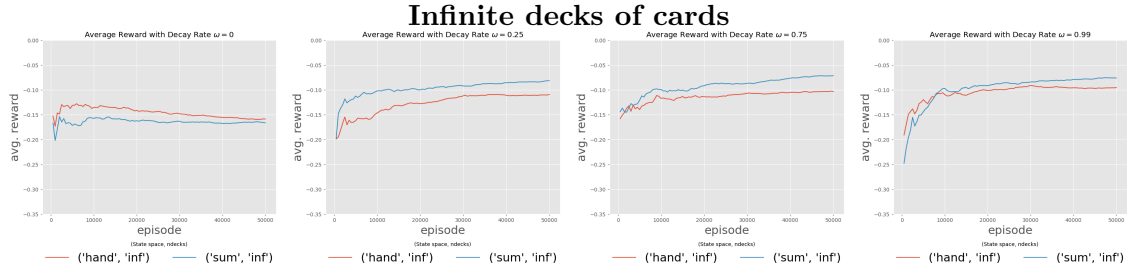


Figure 7: Q-learning with infinite decks, and decay rates $\omega \in \{0, 0.25, 0.75, 0.99\}$. Best model: State-space S_2 ("sum") with decay rate $\omega = 0.75$

As we can see, for most deck sizes we receive the highest average reward with larger values of ω . In particular, $\omega = 0.99$ for the one-deck simulation and $\omega = 0.75$ for the others, generates the highest average reward. For these decay rates, the lesser informative (sum) state-space S_2 seems better for training the Q-learning algorithm. Even though using only 1 deck results in the overall best average reward the difference between different deck sizes does not seem to affect the results significantly. Furthermore, as can be seen in the plots, the learning rate α_n does not affect average reward significantly except when using a constant learning rate $\alpha_n = c$, $\omega = 0$, which decreases the average reward further to about -0.15 , where c in this case is 1. Thus, going forward we choose to continue using the decay rates $\omega = 0.75$ for all simulations but the one-deck simulation, for which we use $\omega = 0.99$.

Constant VS Decaying Exploration Rate

Now we look at what effect the exploration rate ϵ has on the three algorithms. This corresponds to the probability of not choosing the optimal action in the ϵ -greedy update.

The following plots show the average reward using different constant ϵ -values, compared to using a decaying exploration rate, where $\epsilon_n = \frac{\epsilon}{N_n(s,a)+1}$.

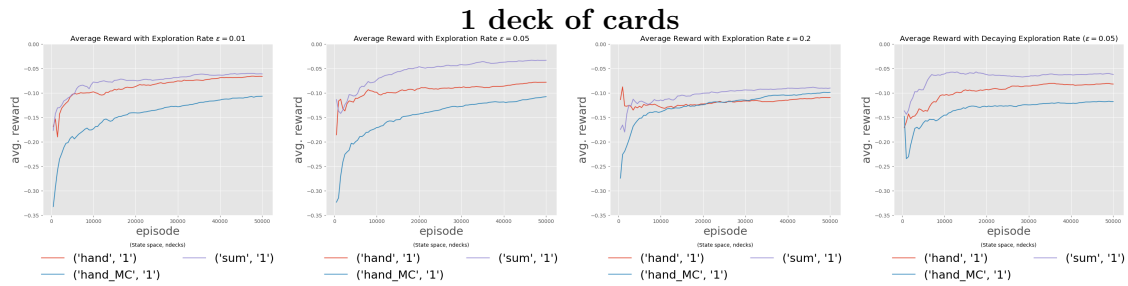


Figure 8: Q-learning and MC with one deck, $\omega = 0.99$, comparing constant explorations with $\epsilon \in \{0.01, 0.05, 0.2\}$ VS decaying exploration with $\epsilon = 0.05$.

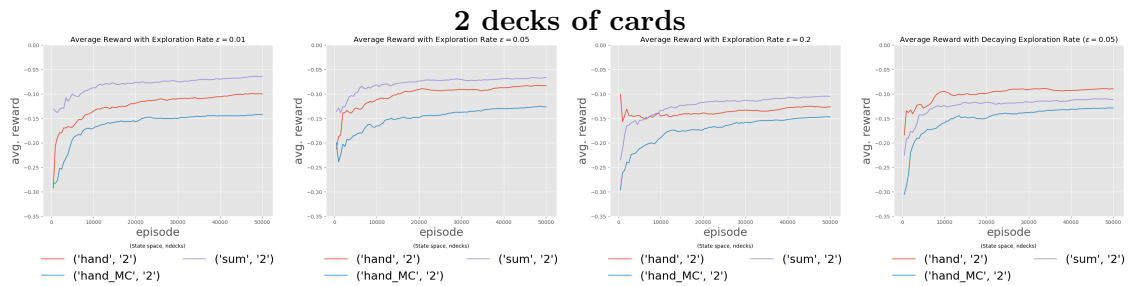


Figure 9: Q-learning and MC with 2 decks, $\omega = 0.75$, comparing constant explorations with $\epsilon \in \{0.01, 0.05, 0.2\}$ VS decaying exploration with $\epsilon = 0.05$.

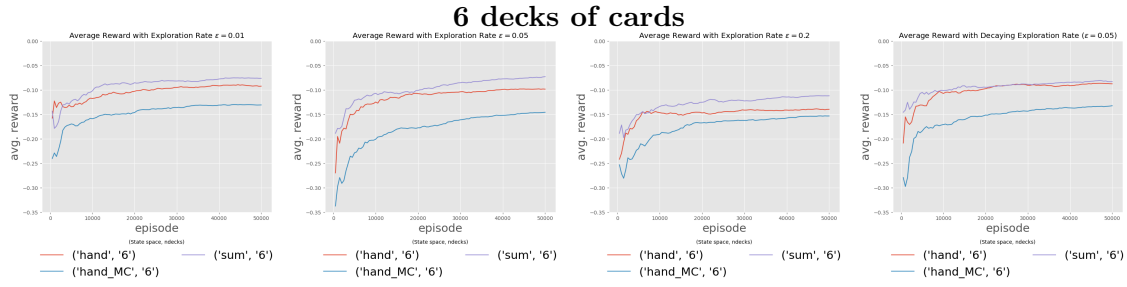


Figure 10: Q-learning and MC with 6 decks, $\omega = 0.75$, comparing constant explorations with $\epsilon \in \{0.01, 0.05, 0.2\}$ VS decaying exploration with $\epsilon = 0.05$.

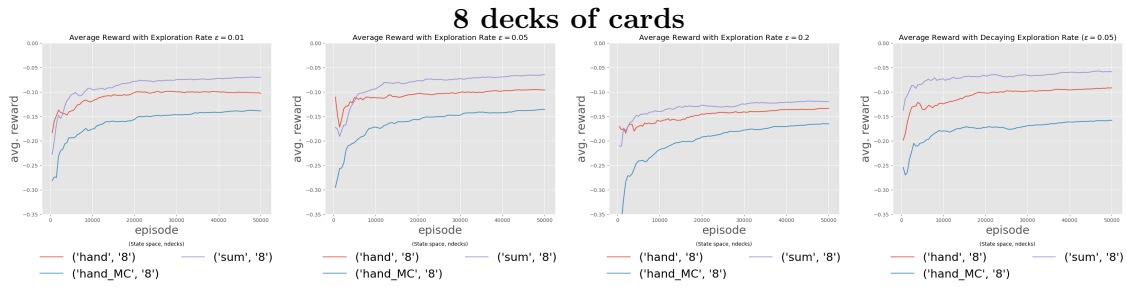


Figure 11: Q-learning and MC with 8 decks, $\omega = 0.75$, comparing constant explorations with $\epsilon \in \{0.01, 0.05, 0.2\}$ VS decaying exploration with $\epsilon = 0.05$.

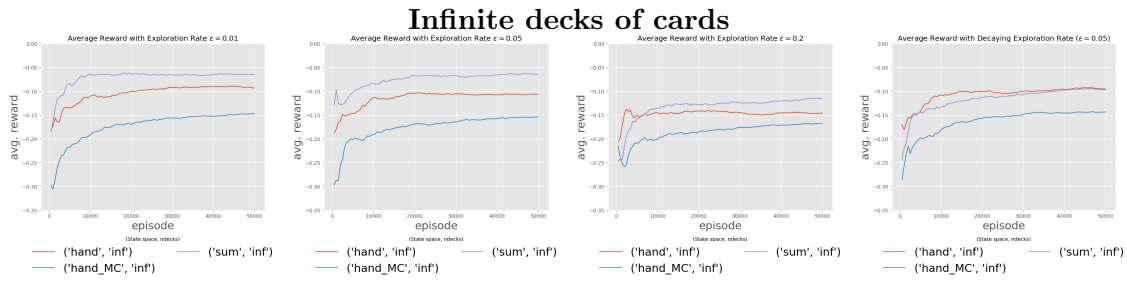


Figure 12: Q-learning and MC with infinite decks, $\omega = 0.75$, comparing constant explorations with $\epsilon \in \{0.01, 0.05, 0.2\}$ VS decaying exploration with $\epsilon = 0.05$.

Exploring different constant exploration rates near $\epsilon = 0.05$, we find that there no value that gives a significantly better model. However, we see that when $\epsilon = 0.2$, the average reward decreases a lot, which was a trend for increasing ϵ as well.

Using 1, 2, 6 and infinite decks, we find that the best model is the Q-learning algorithm

with state-space S_2 ("sum") and constant exploration rate with $\epsilon = 0.05$.

Using 8 decks however, we find the Q-learning model using a decaying exploration with $\epsilon = 0.05$, for the state-space S_2 yields the highest average reward.

The results for each deck are presented in table 1

Decks	ω	ϵ	Decaying (Y/N)	State-Space	Av. Reward	Explored States
1	0.99	0.05	N	S_2	-0.0336	657
2	0.75	0.05	N	S_2	-0.0672	725
6	0.75	0.05	N	S_2	-0.0641	716
8	0.75	0.05	Y	S_2	-0.0618	674
∞	0.75	0.05	N	S_2	-0.0672	721

Table 1: Summary of the most optimal models found, using Q-learning.

Close to optimal Strategy

As discussed, the best algorithm was found to be Q-learning using the S_2 state space representation. This is used to derive close to optimal strategy heatmaps for the agent, which is shown in Figure 13.

The most trivial cases in both instances (which we can use for some sanity check) is when the dealer has strong cards. The agent is more likely to "Hit" against strong dealer cards ($S_d = 10$ or 1) to reduce the dealer's advantage. This is evident in both heatmaps in Figure 13.

Additionally, the heatmaps clearly show that the agent is more inclined to "Hit" when the player sum is low. In the case where the player has a **usable ace**, the general strategy suggests "Hitting" if the player sum is 18 or below. Because of the usable ace, the player can take greater risks even with a larger player sum.

As for the case of **no usable ace**, the strategy is less risk-taking, more often recommending the agent "Stay". Staying is marginally better in the cases where dealer has weak face cards ($S_d = 4, 5, 6$) but hitting becomes more favorable in the cases dealer's face cards get stronger and we take more risk to reduce dealer's increasing advantage. This is due to the dealer having a larger risk of going bust if they start with low cards, resulting in a less aggressive strategy for the agent.

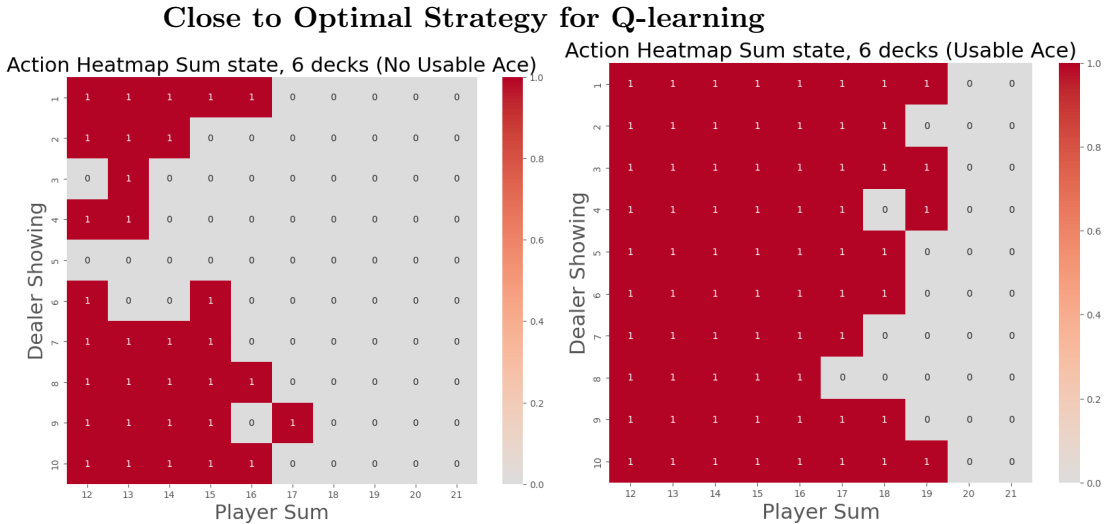


Figure 13: Left: Optimal Strategy Heatmap without Usable Ace. Right: Optimal Strategy Heatmap with Usable Ace. Both heatmaps are derived for 6 decks of cards. 1 (Red) is hit and 0 (Gray) is stay.

In order to get a better understanding of the consequences of the actions, we plot the same heatmaps as in Figure 13, along with the expected reward if the optimal action is taken. This is shown in Figure 14.

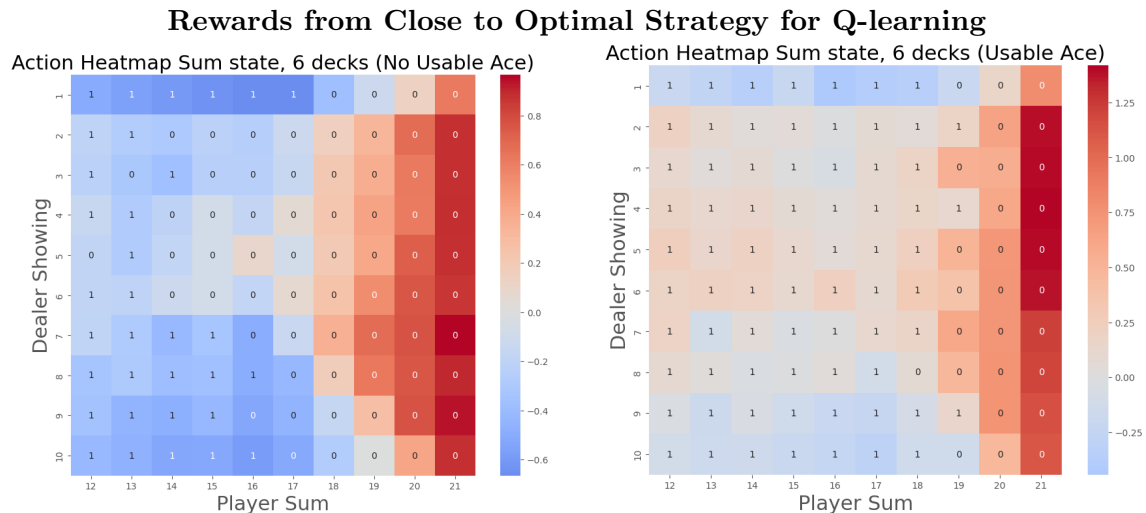


Figure 14: Left: Optimal Strategy Heatmap without Usable Ace. Right: Optimal Strategy Heatmap with Usable Ace. Both heatmaps are derived for 6 decks of cards. 1 is hit and 0 is stay. Intensity of color is action value, i.e. expected reward.

A clear difference between the two cases in Figure 14 is the reduction in blue intensity in the usable ace case. This indicates that the expected losses are less when the agent has the flexibility of treating Ace as 1 or 11, which tips the agent's action to be slightly more risky as apparent by the low intensity 1's.

In general, we see that if the player has **no usable ace** and the dealer shows 1 or 10, the expected loss is high. Despite the agent choosing the optimal strategy, they stand very little chance against the dealer. The expected reward is however positive in many cases where the dealer shows weak cards, and the player sum is above 16.

As for the case with a **usable ace**, the expected reward is still negative if the dealer's cards are strong. However, many more states give positive expected rewards in this case, since the player is more flexible and can keep hitting if they have an ace.

Conclusion

The Q-learning algorithms in both state spaces generally outperformed the Monte Carlo method. When exploration rate is decaying the Q-learning consistently outperformed the MC method, with the state space S_2 model being generally the superior out of the Q-learning models. Additionally, for the Q-learning methods, the state space S_2 model performed better with higher learning rates. For the close to optimal strategy analysis, the key insight is that having a usable ace reduces the expected loss across the board, pointing at the added flexibility of having the option to choose what the value of ace is.

Appendix

Listing 1: RL.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

# import blackjack as bjk
import numpy as np
# import pandas as pd
from collections import defaultdict
import random

def learn_Q(env, n_sims, gamma = 1, omega = 0.77, epsilon = 0.05,
            init_val = 0.0, Q_init = None, episode_file = None,
            warmup=10000):
    """
    gamma: discount factor
    omega: polynomial learning rate parameter (Even-Dar & Mansour, 2003)
    epsilon: exploration probability parameter
    init_val: initiate Q-values to something other than 0?
    Q_init: pre-trained Q dict
    episode_file: save ave
    """

    # Can start with a previously trained Q dict
    if Q_init is None:
        Q = defaultdict(lambda: np.zeros(env.action_space.n) + init_val)
    else:
        Q = Q_init

    state_action_count = defaultdict(lambda: np.zeros(env.action_space.n,
                                                         dtype = int))

    avg_reward = 0.0
    # if we want to save the episode reward to a file,
    if episode_file:
        f = open(episode_file, "w+")
        f.write("episode,avg_reward\n")
    else:
        f = None
    for episode in range(1,n_sims + 1):
        done = False
        action_reward = 0.0
        episode_reward = 0.0
        state = env.reset()
        while not done:
            #Here you cans switch between decaying exploration and constant
            # exploration
            explore = random.random() < epsilon
```

```

        # explore = random.random() < (epsilon / (1 +
            state_action_count[state].sum()))
    if state not in Q or explore:
        # Take a random action
        action = env.action_space.sample()
    else:
        # Take the best possible action
        action = np.argmax(Q[state])

    # Update the state-action count
    state_action_count[state][action] += 1

    # Draw the next state and reward of previous action
    state2, action_reward, done, info = env.step(action)
#####
    # YOUR CODE HERE
    # Compute the learning rate and update the Q-function
    c = 1
    n = state_action_count[state][action]
    alpha = c*n*(-omega)
    Q[state][action] += alpha*(action_reward + gamma*np.max(Q[
        state2]) - Q[state][action])

#####
    # Update state and episode reward
    state = state2
    episode_reward += action_reward

    if episode % (n_sims // 100) == 0:
        #print('Mean avg reward, after {} episodes: {}'.format(
            #episode, avg_reward))
        if f:
            # append to the file which we want to save to
            f.write("{}{}\n".format(episode, str(avg_reward)))

    # Game is over
    avg_reward += (episode_reward - avg_reward) / (episode + 1)
    return Q, avg_reward, state_action_count

def Q_policy(state, Q, env):
    if state in Q:
        return np.argmax(Q[state])
    return env.action_space.sample()

def filter_states(S):
    return {k: v for k, v in S.items() if type(k) == tuple and \
        k[0] > 11 and k[0] < 22 and k[1] < 11}

```

```

def fill_missing_sum_states(D, default_value = 0.0):
    S = D
    for player_sum in range(12, 22):
        for dealer_sum in range(1, 11):
            state0 = (player_sum, dealer_sum, False)
            state1 = (player_sum, dealer_sum, True)
            if state0 not in S:
                S[state0] = default_value
            if state1 not in S:
                S[state1] = default_value
    return S

def convert_to_sum_states(Q, env):
    """
    Function which convert the expanded state spce to a sum-based state
    space
    """
    S = dict()
    n = defaultdict(int)
    use_ace = lambda x: x[0] > 0 and \
        np.dot(env.deck_values, x) + 10 <= 21
    sum_p = lambda x: np.dot(env.deck_values, x) + \
        10 * use_ace(x)
    for state, action_values in Q.items():
        sum_state = (sum_p(state[0]), state[1],
                    use_ace(state[0]))
        if sum_state in S:
            n[sum_state] += 1
            S[sum_state] = (action_values + n[sum_state] * S[sum_state]) \
                / (n[sum_state] + 1)
        else:
            S[sum_state] = action_values
    return S

def convert_to_value_function(Q):
    S = dict()
    for state, action_values in Q.items():
        S[state] = action_values.max()
    return S

def learn_MC(env, n_sims, gamma = 1, epsilon = 0.05,
             init_val = 0.0, Q_init = None, episode_file = None,
             warmup=10000):
    """
    gamma: discount factor
    epsilon: exploration probability parameter
    init_val: initiate Q-values to something other than 0?
    Q_init: pre-trained Q dict
    """

```

```

episode_file: save ave
"""

# Can start with a previously trained Q dict
if Q_init is None:
    Q = defaultdict(lambda: np.zeros(env.action_space.n) + init_val)
else:
    Q = Q_init

state_action_count = defaultdict(lambda: np.zeros(env.action_space.n,
                                                    dtype = int))

avg_reward = 0.0
# if we want to save the episode reward to a file,
if episode_file:
    f = open(episode_file, "w+")
    f.write("episode, avg_reward\n")
else:
    f = None
for episode in range(1, n_sims + 1):
    done = False
    action_reward = 0.0
    episode_reward = 0.0
    state = env.reset()
    episode_state_action_count = defaultdict(lambda: np.zeros(env.
                                                                action_space.n,
                                                                dtype = int))

    episode_track = [] ### Saving the tuple (state, action, reward) to
        create G from the rewards

    G = defaultdict(lambda: np.zeros(env.action_space.n, dtype = float)
                    )

    j = 0
    while not done:
        j += 1
        #Here you cans switch between decaying exploration and constant
        exploration
        explore = random.random() < epsilon
        # explore = random.random() < (epsilon / (1 +
            state_action_count[state].sum()))
        if state not in Q or explore:
            # Take a random action
            action = env.action_space.sample()
        else:
            # Take the best possible action
            action = np.argmax(Q[state])

        # Update the state-action count
        state_action_count[state][action] += 1
        episode_state_action_count[state][action] += 1

```

```

        # Draw the next state and reward of previous action
        state2, action_reward, done, info = env.step(action)
        state = state2
        episode_reward += action_reward
        episode_track.append((state, action, action_reward))

    if episode % (n_sims // 100) == 0:
        #print('Mean avg reward, after {} episodes: {}'.format(
            #episode, avg_reward))
        if f:
            # append to the file which we want to save to
            f.write("{}{}\n".format(episode, str(avg_reward)))
        # Game (episode) is over
        #####
        # YOUR CODE HERE

        # Update avg_reward
        avg_reward = avg_reward + 1/episode*(episode_reward - avg_reward)

        # Compute G(s, a) and update Q-values
        visited = set()
        for i, (state, action, _) in enumerate(episode_track):
            if (state, action) not in visited:
                first_idx = i
                visited.add((state, action))
                G[state][action] = sum(gamma**k * reward for k, (_, _,
                    reward) in enumerate(episode_track[first_idx:]))

        # Update the Q-function for each visited state/action pair.
        for state in episode_state_action_count:
            for action in range(env.action_space.n):
                if episode_state_action_count[state][action] > 0:
                    Q[state][action] += (G[state][action] - Q[state][action]
                        )/state_action_count[state][action]

        #####
    return Q, avg_reward, state_action_count

```

Listing 2: plotting.py

```
# code copied from: https://github.com/dennybritz/reinforcement-learning/
blob/master/lib/plotting.py

import matplotlib
import numpy as np
import pandas as pd
from collections import namedtuple
from matplotlib import pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns

def plot_value_function(V, title="Value Function",
                       directory = None, file_name = None,
                       show = False):
    """
    Plots the value function as a surface plot.

    code copied from: https://github.com/dennybritz/reinforcement-learning/
    blob/master/lib/plotting.py
    """
    min_x = min(k[0] for k in V.keys())
    max_x = max(k[0] for k in V.keys())
    min_y = min(k[1] for k in V.keys())
    max_y = max(k[1] for k in V.keys())

    x_range = np.arange(min_x, max_x + 1)
    y_range = np.arange(min_y, max_y + 1)
    X, Y = np.meshgrid(x_range, y_range)

    # Find value for all (x, y) coordinates
    Z_noace = np.apply_along_axis(lambda _: V[(_[0], _[1], False)], 2,
                                  np.dstack([X, Y]))
    Z_ace = np.apply_along_axis(lambda _: V[(_[0], _[1], True)], 2,
                                 np.dstack([X, Y]))

    def plot_surface(X, Y, Z, title, usable_ace):
        fig = plt.figure(figsize=(20, 10))
        ax = fig.add_subplot(111, projection='3d')
        surf = ax.plot_surface(X, Y, Z, rstride=1, cstride=1,
                               cmap=matplotlib.cm.coolwarm, vmin=-1.0, vmax=
                               =1.5)
        ax.set_xlabel('Player Sum', size=22)
        ax.set_ylabel('Dealer Showing', size=22)
        ax.set_zlabel('Value', size=22)
        ax.set_title(title, size=22)
        ax.view_init(ax.elev, -120)
        fig.colorbar(surf)
        if file_name:
```

```

        fig.savefig(directory + usable_ace + file_name, bbox_inches = '
            tight')
    if show:
        fig.show()
    plt.close(fig)

def plot_value_function_2(Q, V, title="Value_Function",
    directory = None, file_name = None,
    show = False):
    """
    Plots the value function as a surface plot.

    code copied from: https://github.com/dennybritz/reinforcement-learning/
    blob/master/lib/plotting.py
    """
    min_x = min(k[0] for k in V.keys())
    max_x = max(k[0] for k in V.keys())
    min_y = min(k[1] for k in V.keys())
    max_y = max(k[1] for k in V.keys())

    x_range = np.arange(min_x, max_x + 1)
    y_range = np.arange(min_y, max_y + 1)
    X, Y = np.meshgrid(x_range, y_range)

    # Compute action and max Q-value grids for hit/stay, A for action (0,
    1) and C for value (intensity of colour in the plot)
    A_noace = np.apply_along_axis(
        lambda _: np.argmax(Q[(_[0], _[1], False)]), 2, np.dstack([X, Y])
    )

    C_noace = np.apply_along_axis(
        lambda _: np.max(Q[(_[0], _[1], False)]), 2, np.dstack([X, Y])
    )

    A_ace = np.apply_along_axis(
        lambda _: np.argmax(Q[(_[0], _[1], True)]), 2, np.dstack([X, Y])
    )

    C_ace = np.apply_along_axis(
        lambda _: np.max(Q[(_[0], _[1], True)]), 2, np.dstack([X, Y])
    )

def plot_heatmap(A, C, title, usable_ace):
    """
    Plots a heatmap of the close to optimal strategy for the algorithms

    Color = max(Q), Value = best action (0=Hit, 1=Stay)
    """
    fig, ax = plt.subplots(figsize=(10, 8))

```

```

sns.heatmap(C, annot=A, fmt="d", cmap="coolwarm", center=0,
            xticklabels=x_range, yticklabels=y_range, cbar=True)

ax.set_xlabel('Player Sum', size=22)
ax.set_ylabel('Dealer Showing', size=22)
ax.set_title(title, size=22)

if file_name:
    fig.savefig(f"{directory}ActionHeatMap_{usable_ace}{file_name}"
                , bbox_inches='tight')

if show:
    plt.show()
plt.close(fig)

plot_heatmap(A_noace, C_noace, "ActionHeatmap_{ }(NoUsableAce)".
            format(title),
            usable_ace = "noace_")
plot_heatmap(A_ace, C_ace, "ActionHeatmap_{ }(UsableAce)".format(
            title),
            usable_ace = "ace_")

def plot_avg_reward_episode(path, env_types, ndecks):
    """
    Function which plots the average return over episodes

    path: path to the folder where the data resides
    env_types: list with the env types you want to plot
    ndecks: a list with the decks that you want to plot
    save_path: optional path of where to save the fig.
    """
    def load_df(path, env_type, ndecks):
        assert env_type in ["hand_MC", "hand", "sum"]
        path_to_file = "{}/{}_state_{}.txt".format(path, env_type, ndecks)
        df = pd.read_table(path_to_file, sep=",")
        df['env_type'] = env_type
        df['ndecks'] = ndecks
        return df

    df_l = []
    for env in env_types:
        for deck in ndecks:
            df_l.append(load_df(path, env, deck))
    df = pd.concat(df_l)

    fig, ax = plt.subplots(figsize=(8,6))
    lab= []
    for label, df in df.groupby(["env_type", "ndecks"]):
        lab.append(label)
        ax.plot(df['episode'], df['avg_reward'], label=label)

```

```

lgd = ax.legend(title="(State space, undecks)", loc='upper center',
                bbox_to_anchor=(0.5, -0.1),
                shadow=False, ncol=2, framealpha=0.0, fontsize=22)
ax.set_xlabel("episode", size=22)
ax.set_ylabel("avg. reward", size=22)
return fig, lgd

```